Natural Language Processing (NLP)

Text processing, representations, classic NLP tasks, and the road to transformers.

Contents

#	Topic	Level
1	Text preprocessing: tokenisation, lowercasing, stop words, stemming, lemmatisation	Beginner
2	Bag of Words, TF-IDF and n-grams	Beginner
3	Text classification with classical ML	Beginner
4	Word embeddings: Word2Vec, GloVe, FastText	Intermediate
5	Language models: n-gram, neural LM, perplexity	Intermediate
6	Sequence labelling: POS tagging, NER, chunking	Intermediate
7	Parsing, coreference, semantic similarity, topic modelling (LDA, BERTopic)	Intermediate
8	Machine translation, summarisation, question answering	Advanced
9	Tokenisation for transformers: BPE, WordPiece, SentencePiece	Intermediate
10	BERT family and fine-tuning for NLP tasks	Advanced
11	GPT-style decoders and T5/BART encoder-decoders	Advanced
12	Sentence embeddings and semantic search	Intermediate
13	Evaluation: BLEU, ROUGE, METEOR, BERTScore, perplexity, human eval	Advanced
14	Multilingual and Indian-language NLP (Tamil, Hindi)	Intermediate
15	Speech: ASR (Whisper), TTS and voice assistants	Advanced

1. Text preprocessing: tokenisation, lowercasing, stop words, stemming, lemmatisation [Beginner]

Explanation (English)

Break text into tokens, normalise, remove noise; stemming chops suffixes, lemmatisation returns dictionary form.

Tanglish (Tamil + English) Vilakkam

Text clean: tokenize, stopwords remove, stemming/lemmatization.

Example / Program

```
import re
t=re.sub(r'[^a-z ]','','Hello, NLP World!'.lower()).split();print(t)
```

Learn more

YouTube search: Text preprocessing: tokenisation, lowercasing, stop words, stemming, lemmatisation tutorial

2. Bag of Words, TF-IDF and n-grams [Beginner]

Explanation (English)

Represent documents as sparse word-count vectors; TF-IDF down-weights common words; n-grams capture local order.

Tanglish (Tamil + English) Vilakkam

BoW, TF-IDF - text-a numbers-a maathum.

Example / Program

```
from sklearn.feature_extraction.text import TfidfVectorizer
v=TfidfVectorizer();X=v.fit_transform(['ai is fun','ml is fun too']);print(v.get_feature_names_out(),X.toarray().round(2)
```

Learn more

YouTube search: Bag of Words, TF-IDF and n-grams tutorial

3. Text classification with classical ML [Beginner]

Explanation (English)

TF-IDF + Logistic Regression / Naive Bayes / SVM is a strong baseline for sentiment, spam and topic tagging.

Tanglish (Tamil + English) Vilakkam

TF-IDF + Logistic Regression strong baseline.

Example / Program

```
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
m=make_pipeline(TfidfVectorizer(),LogisticRegression()).fit(['good','bad','great','awful'],[1,0,1,0]);print(m.predict(['g
```

Learn more

YouTube search: Text classification with classical ML tutorial

4. Word embeddings: Word2Vec, GloVe, FastText [Intermediate]

Explanation (English)

Dense vectors where similar words are close; king - man + woman ~ queen. Static (one vector per word).

Tanglish (Tamil + English) Vilakkam

Word2Vec: words-a vectors-a maathum. Similar words nearby.

Example / Program

```
from gensim.models import Word2Vec
m=Word2Vec([['i','love','ai'],['i','love','ml']]*50,vector_size=20,min_count=1);print(m.wv.most_similar('ai')[:2])
```

Learn more

YouTube search: Word embeddings: Word2Vec, GloVe, FastText tutorial

5. Language models: n-gram, neural LM, perplexity [Intermediate]

Explanation (English)
Predict the next token; perplexity measures how surprised the model is. Neural LMs replaced n-grams.

Tanglish (Tamil + English) Vilakkam
Language model = next word predict.

Example / Program
```
import collections
bi=collections.Counter(zip('abab'.split(),'abab'.split()[1:]))
```

Learn more
YouTube search: Language models: n-gram, neural LM, perplexity tutorial

6. Sequence labelling: POS tagging, NER, chunking [Intermediate]

Explanation (English)
Assign a label per token (CRF, BiLSTM-CRF, transformers).

Tanglish (Tamil + English) Vilakkam
NER: names, places, orgs identify.

Example / Program
```
import spacy;nlp=spacy.load('en_core_web_sm');print([(t.text,t.pos_) for t in nlp('AI is great')])
```

Learn more
YouTube search: Sequence labelling: POS tagging, NER, chunking tutorial

7. Parsing, coreference, semantic similarity, topic modelling (LDA, BERTopic) [Intermediate]

Explanation (English)
Syntactic dependency parsing; resolving pronouns; embedding similarity; discovering topics unsupervised.

Tanglish (Tamil + English) Vilakkam
Topic modelling: LDA, BERTopic.

Example / Program
```
from sklearn.decomposition import LatentDirichletAllocation
from sklearn.feature_extraction.text import CountVectorizer
X=CountVectorizer().fit_transform(['cat dog','dog bone','stock market','market price']);print(LatentDirichletAllocation(2
```

Learn more
YouTube search: Parsing, coreference, semantic similarity, topic modelling (LDA, BERTopic) tutorial

8. Machine translation, summarisation, question answering [Advanced]

Explanation (English)
Seq2seq and Transformer models (MarianMT, T5, BART, mT5). Extractive vs abstractive summarisation; extractive vs generative QA.

Tanglish (Tamil + English) Vilakkam
Translation, summary, QA tasks.

Example / Program
```
from transformers import pipeline
print(pipeline('summarization')('Long text ... '*30,max_length=30))
```

Learn more
YouTube search: Machine translation, summarisation, question answering tutorial

9. Tokenisation for transformers: BPE, WordPiece, SentencePiece [Intermediate]

Explanation (English)
Subword tokenisers handle rare words and any language. Vocabulary is learnt from data.

Tanglish (Tamil + English) Vilakkam
Subword tokenization - BPE, WordPiece.

Example / Program
```
from transformers import AutoTokenizer
t=AutoTokenizer.from_pretrained('bert-base-uncased');print(t.tokenize('unbelievable'))
```

Learn more
YouTube search: Tokenisation for transformers: BPE, WordPiece, SentencePiece tutorial

10. BERT family and fine-tuning for NLP tasks [Advanced]

Explanation (English)
Encoder-only masked language models (BERT, RoBERTa, DistilBERT, DeBERTa). Fine-tune with a classification head for sentiment, NER, QA.

Tanglish (Tamil + English) Vilakkam
BERT bidirectional encoder. Fine-tune pannalam.

Example / Program
```
from transformers import AutoModelForSequenceClassification as M,AutoTokenizer as Tk
t=Tk.from_pretrained('distilbert-base-uncased');m=M.from_pretrained('distilbert-base-uncased',num_labels=2)
print(m(**t('hello',return_tensors='pt')).logits)
```

Learn more
YouTube search: BERT family and fine-tuning for NLP tasks tutorial

11. GPT-style decoders and T5/BART encoder-decoders [Advanced]

Explanation (English)
Decoder-only autoregressive models generate text; encoder-decoder models suit translation/summarisation.

Tanglish (Tamil + English) Vilakkam
GPT = decoder, T5/BART = encoder-decoder.

Example / Program
```
from transformers import pipeline
g=pipeline('text-generation','gpt2');print(g('AI will',max_new_tokens=20)[0]['generated_text'])
```

Learn more
YouTube search: GPT-style decoders and T5/BART encoder-decoders tutorial

12. Sentence embeddings and semantic search [Intermediate]

Explanation (English)
Encode sentences into vectors with sentence-transformers and search by cosine similarity - the basis of RAG.

Tanglish (Tamil + English) Vilakkam
Semantic search: embeddings + cosine similarity.

Example / Program
```
from sentence_transformers import SentenceTransformer,util
m=SentenceTransformer('all-MiniLM-L6-v2')
e=m.encode(['cat on mat','dog in yard','stock prices']);print(util.cos_sim(e[0],e))
```

Learn more
YouTube search: Sentence embeddings and semantic search tutorial

13. Evaluation: BLEU, ROUGE, METEOR, BERTScore, perplexity, human eval [Advanced]

Explanation (English)

Overlap metrics for generation, embedding-based metrics, and human or LLM-as-judge evaluation.

Tanglish (Tamil + English) Vilakkam

BLEU/ROUGE generation evaluate panna.

Example / Program

```
from nltk.translate.bleu_score import sentence_bleu
print(sentence_bleu([['the','cat']],['the','cat']))
```

Learn more

YouTube search: Evaluation: BLEU, ROUGE, METEOR, BERTScore, perplexity, human eval tutorial

14. Multilingual and Indian-language NLP (Tamil, Hindi) [Intermediate]

Explanation (English)

Use multilingual models (mBERT, XLM-R, IndicBERT, MuRIL, AI4Bharat IndicTrans2). Handle Unicode, transliteration and code-mixed (Tanglish) text.

Tanglish (Tamil + English) Vilakkam

Tamil NLP: IndicBERT, IndicTrans2, AI4Bharat. Tanglish code-mixed text special.

Example / Program

```
from transformers import pipeline
# pipeline('fill-mask','xlm-roberta-base')('Chennai is the <mask> of Tamil Nadu.')
```

Learn more

YouTube search: Multilingual and Indian-language NLP (Tamil, Hindi) tutorial
AI4Bharat: https://ai4bharat.iitm.ac.in

15. Speech: ASR (Whisper), TTS and voice assistants [Advanced]

Explanation (English)

Speech-to-text with Whisper/wav2vec2; text-to-speech with VITS/XTTS; pipelines for voice bots.

Tanglish (Tamil + English) Vilakkam

Whisper speech-to-text; TTS.

Example / Program

```
import whisper
# print(whisper.load_model('base').transcribe('a.mp3')['text'])
```

Learn more

YouTube search: Speech: ASR (Whisper), TTS and voice assistants tutorial